

Code Review — solar-sync-broker-api

Code Review — solar-sync-broker-api (pass 3)

Branch: feat/fundatie · Vertical-slice Spring Boot 3.5 · Java 21 Slices implementate: auth, house, battery, users, system Teste: 32 verzi (unit + IT) — dar niciun test nu trece prin HTTP pe slice-ul battery, deci problemele 🚫 de mai jos **nu sunt prinse de teste**.

Legendă severitate

- 🚫 Critice (B1, B2...) — bug-uri, securitate, endpoint-uri moarte
- 🟡 Importante (M1, M2...) — design, pattern greșit
- 🟢 Cleanups (C1, C2...) — stil, naming

Reparat de la ultimul pass ✅

- getHouseById ia acum identitatea din Principal, nu din query param — gaura principală de ownership e închisă (HouseController.java:40).
- UserController are constructor injection (nu mai e NPE garantat) și @PreAuthorize (dar vezi B4).
- BatteryController există acum — slice-ul battery e expus prin HTTP (dar vezi B2).
- GlobalExceptionHandler întoarce 404 pe wire pentru "not found" (dar vezi B3).
- House.equals / hashCode reduce la id + name ; 401/403 ies ca JSON real prin entry point + access denied handler.

🚫 Critice

B1. Oricine se poate înregistra ca ADMIN — privilege escalation

AuthRegisterRequest.java:12 — userType vine **de la client**, iar AuthServiceImpl.java:55 îl salvează ca atare. permissionsForType (ServiceImpl.java:99) dă ADMIN-ului house:manage + battery:manage.

Deci fluxul de atac e: POST /api/v1/auth/register cu "userType": "ADMIN" → primești toate permisiunile → poți crea/șterge orice casă. Endpoint-ul e public (/api/v1/auth/** e permitAll), deci nu e nevoie de niciun cont existent.

Mecanismul: rolul e o decizie a *serverului*, nu o preferință a *clientului*. Tot ce vine în request body e input nesigur — inclusiv câmpuri "administrative". Register-ul public trebuie să forțeze UserType.USER ; ADMIN se creează prin seed sau printr-un endpoint protejat de un ADMIN existent.

B2. Tot slice-ul battery e mort prin HTTP — authority-urile nu se potrivesc

BatteryController.java:27 cere hasAuthority('BATTERY_VIEW'), iar BatteryController.java:34 cere hasAuthority('BATTERY_CONFIG').

Dar authority-urile reale din token sunt **string-urile permisiunilor**, nu numele enum-ului: User.java:66–68 mapează p.getPermission() → "battery:view", "battery:manage" ... iar BATTERY_CONFIG **nici nu există** în UserPermissions.java:7–10.

Rezultat: GET /houses/{id}/battery → **403 pentru toată lumea, inclusiv ADMIN**. La fel PATCH ../battery/configuration. Testul unit (BatteryQueryServiceTests) apelează serviciul direct, deci sare peste stratul de securitate și nu prinde asta — de-asta un slice are nevoie de măcar un IT prin MockMvc cu authorities reale.

Mecanismul: hasAuthority(X) face string-match exact pe ce a pus getAuthorities() în token. Enum-ul are două identități — numele constantei (BATTERY_VIEW) și valoarea (battery:view) — și trebuie ales **unul singur** peste tot. Restul codului folosește deja valoarea (house:view în HouseController), deci aici trebuie battery:view / battery:manage.

B3. GlobalExceptionHandler — body null , status intern greșit, excepții nemapate

GlobalExceptionHandler.java:23-27 :

```
ApiResponse apiErrorResponse = ApiResponse.builder()
    .message(e.getMessage()).status(HttpStatus.NO_CONTENT.value()).build();
return new ResponseEntity<>(apiErrorResponse.error(), HttpStatus.NOT_FOUND);
```

Trei probleme într-un singur return:

1. `apiErrorResponse.error()` — câmpul `error` **nu e setat niciodată** de builder → clientul primește 404 cu `body null`. Tot obiectul construit (`message`, `status`) e aruncat.
2. `status(HttpStatus.NO_CONTENT.value())` — 204 în interiorul unui răspuns 404 (chiar dacă acum nu se vede, pentru că `body-ul` e `null`).
3. Același pattern la `handleAlreadyExistsExceptions` (`GlobalExceptionHandler.java:41`).

Și **excepțiile nemapate** cad pe 500:

- `HouseAccessDeniedHandler` (aruncată în `HouseQueryServiceImpl.java:45` și `BatteryQueryServiceImpl.java:32`) → ar trebui **403**, azi e **500**. Deci exact scenariul "user străin încearcă casa altuia" — cel reparat în B1 din pass-ul trecut — răspunde cu 500 în loc de 403.
- `IllegalArgumentException` din `HouseQueryServiceImpl.java:41` → 500.

Mecanismul: ai deja soluția în proiect — `ApiResponse.of(...)` populează toate câmpurile, iar entry point-ul + access denied handler-ul o folosesc corect. Handler-ul global trebuie să întoarcă `ResponseEntity<ApiResponse>` (Jackson îl serializează), nu `ResponseEntity<String>` cu un câmp scos manual.

B4. UserController./add — endpoint mort + user invalid dacă ar fi viu

Două straturi de probleme:

- `UserController.java:28` — `hasAuthority('ADMIN')`: nu există niciun authority `ADMIN` în token (vezi B2 — token-ul conține doar `house:view` etc., iar `userType` nu e pus ca authority) → **403 pentru toată lumea**, endpoint mort.
- Dacă ar fi viu: `UserCommandServiceImpl.java:25-29` construiește user **fără password și fără userType** — ambele coloane sunt `nullable = false` (`User.java:49,53`) → `ConstraintViolationException` → 500. În plus `save` e apelat de **două ori** (`UserCommandServiceImpl.java:31-32`) și `UserController.java:31` face `.build()` fără `body`, aruncând `UserResponse`-ul calculat.
- `UserRequest.java:3` importă `org.hibernate.usertype.UserType` — import mort rămas din versiunea veche.

Fix recomandat: ori scoți controllerul (register-ul acoperă crearea de useri), ori îl faci "admin creates user" pe bune: `UserRequest` cu `parolă` + `tip`, `encodare` cu `PasswordEncoder`, `permisiuni` prin `permissionsForType`, un singur `save`, `body` întors.

B5. Strategia de schemă contrazice backlog-ul (neschimbat din pass-ul 2)

- `application.yml:14` → `ddl-auto: create` — **recrează schema și pierzi datele la fiecare pornire**.
- `application.yml:19-20` → `flyway.enabled: false`, deși `db/migration/` există (gol) și `pom.xml` aduce `flyway-core` + `flyway-mysql`.
- Numele DB diferă între surse: `application.yml:6` → `solarSyncBroker-api`, dar `docker-compose.yml` → `MYSQL_DATABASE: solarsync`. Merge azi doar datorită `createDatabaseIfNotExist=true` + `user root` — adică app-ul își creează singur o DB paralelă cu cea din `compose`.

Fix: migrări Flyway + `ddl-auto: validate`; un singur nume de DB peste tot; conectare cu userul `solarsync`, nu `root`.

Importante

#	Problemă	Unde
M1	<code>getAllHouses</code> întoarce toate casele oricărui user cu <code>house:view</code> (adică oricui) — non-admin ar trebui să-și vadă doar casele lui. Ironia: <code>findByOwnerId</code> / <code>findByIdAndOwnerId</code> există deja și sunt nefolosite	<code>HouseQueryServiceImpl.java:28</code> , <code>HouseRepository.java:13-14</code>
M2	<code>Battery</code> lipsă → <code>HouseNotFoundException</code> (tip greșit; <code>BatteryNotFoundException</code> există); iar <code>updateConfiguration</code> nu are niciun ownership check — după fix-ul B2, orice user cu <code>battery:manage</code> ar configura bateria oricui	<code>BatteryQueryServiceImpl.java:28</code> , <code>BatteryCommandServiceImpl.java:23-36</code>
M3	<code>permissionsForType</code> : <code>OPERATOR</code> și <code>AGENT</code> cad pe <code>else</code> → identici cu <code>USER</code> ; ori primesc seturi proprii, ori nu există încă în enum	<code>AuthServiceImpl.java:99-112</code>
M4	<code>UserResponse</code> e folosit și la register și la login, cu câmp <code>now</code> (nume fără sens) și <code>token</code> null la register; <code>AuthLoginResponse</code> — DTO-ul dedicat — există și e nefolosit	<code>UserResponse.java:16-17</code> , <code>AuthLoginResponse.java</code>
M5	<code>SecurityConstants</code> : <code>PUBLIC_URLS</code> conține string-ul invalid " <code>http://localhost:8080/login#/"</code> (cu spațiu); <code>ISSUER</code> / <code>AUDIENCE</code> = <code>controllerPractice-</code> * (copy-paste din alt proiect); <code>publicAwareBearerTokenResolver</code> e efectiv cod mort din cauza lor	<code>SecurityConstants.java:13-18</code> , <code>SecurityConfiguration.java:89-99</code>
M6	<code>JWTTokenProvider</code> : <code>getAuthorities</code> / <code>isTokenValid</code> / <code>getSubject</code> / <code>getAuthentication</code> / <code>getClaimsFromToken</code> = cod mort — validarea o face resource server-ul prin <code>NimbusJwtDecoder</code>	<code>JWTTokenProvider.java:60-102</code>
M7	<code>User.equals</code> / <code>hashCode</code> includ toate câmpurile (inclusiv <code>password</code> și colecția <code>permissions</code>); <code>Battery</code> include <code>house</code> (asociere LAZY → poate declanșa lazy-loading în <code>hashCode</code>). Modelul corect e deja în <code>House</code> : doar <code>id</code> + business key	<code>User.java:89-100</code> , <code>Battery.java:40-50</code>
M8	<code>updateUser</code> / <code>patchUser</code> întorc null , <code>deleteUser</code> e gol — stub-uri care compilează dar mint; mai bine <code>UnsupportedOperationException</code> sau deloc în interfață	<code>UserCommandServiceImpl.java:36-52</code>
M9	<code>@EnableJpaAuditing</code> + <code>AbstractAuditable</code> există, dar niciun entity nu extinde clasa → auditing-ul nu face nimic	<code>PersistenceConfig.java</code> , <code>AbstractAuditable.java</code>
M10	Stereotipuri inconsistente: <code>@Service</code> pe command-uri , <code>@Component</code> pe query-uri și pe <code>BatteryCommandServiceImpl</code> — toate sunt servicii	<code>HouseQueryServiceImpl.java:16</code> , <code>BatteryQueryServiceImpl.java:15</code> , <code>BatteryCommandServiceImpl.java:13</code>
M11	<code>pom.xml</code> aduce <code>Testcontainers</code> + <code>MySQL</code> pentru teste, dar toate IT-urile rulează pe H2 <code>MODE=MySQL</code> — dialectul diferă de producție și dependențele stau nefolosite	<code>pom.xml</code> , <code>application-test.yml</code>

Cleanups

- C1 — Typos în identificatori publici** (un examinator de licență le vede imediat): `AuthRequisterRequest` , `AuthLoginrequest` , `HouseAlreadyExistsExcption` , `UserAlreadyexistsException` , `JWTAuthenticationEntryPoint` , `efficientyPercent` , `@Table(name="bateries")` (`Battery.java:11`) , plus în teste: `testGetHouseByIdReturnsUnotharized` , `...WithoughtAuthorisation` .
- C2 — Debug uitat**: `System.out.println(principal)` în `HouseController.java:41` — folosește `log.debug(...)` , ai deja `@Slf4j` pe clasă.
- C3 — Importuri moarte**: `org.hibernate.usertype.UserType` în `UserRequest.java:3` ; `jdk.jfr.Timestamp` în `BatteryCommandServiceImpl.java:3` (JFR n-are ce căuta aici).
- C4 — Comentarii adresate reviewer-ului**, nu cititorului: `// Schimbat din @Component în @Service` (`HouseCommandServiceImpl.java:16`) , `// Corectat importul...` (`AuthRequisterRequest.java:8`) , `// Sincronizat la v1` , `// Eliminat cast-ul redundant` (`SecurityConfiguration.java:73,79`) . Astfel de note țin de mesajul de commit, nu de cod.
- C5 — Nume care mint**: `HouseAccessDeniedHandler` e un `RuntimeException` , nu un handler → `HouseAccessDeniedException` ; `UserResponse.now` → `issuedAt` sau `scos` ; `@NoArgsConstructor` + `@NotBlank` pe clasa `SecurityConstants` nu au niciun efect

acolo.

Before / After (criticele)

B1 — register forțează rolul

```
// Before — AuthServiceImpl.register
.userType(authReqisterRequest.userType())           // clientul își alege rolul

// After
.userType(UserType.USER)                             // register public = USER, mereu
// (ADMIN vine din seed sau dintr-un endpoint protejat de un ADMIN existent;
// scoate câmpul userType din AuthReqisterRequest)
```

B2 — authority-uri aliniate la valorile din token

```
// Before — BatteryController
@PreAuthorize("hasAuthority('BATTERY_VIEW')")         // nu există în token
@PreAuthorize("hasAuthority('BATTERY_CONFIG')")        // nu există deloc

// After
@PreAuthorize("hasAuthority('battery:view')")
@PreAuthorize("hasAuthority('battery:manage')")
```

B3 — handler care întoarce obiectul, cu statusul corect

```
// Before
ApiResponse apiErrorResponse = ApiResponse.builder()
    .message(e.getMessage()).status(HttpStatus.NO_CONTENT.value()).build();
return new ResponseEntity<>(apiErrorResponse.error(), HttpStatus.NOT_FOUND); // body null

// After
@ExceptionHandler({UserNotFoundException.class, BatteryNotFoundException.class, HouseNotFoundException.class})
public ResponseEntity<ApiResponse> handleNotFound(RuntimeException e, HttpServletRequest req) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(ApiErrorResponse.of(404, "NOT_FOUND", e.getMessage(), req.getRequestURI()));
}

@ExceptionHandler(HouseAccessDeniedHandler.class) // azi cade pe 500
public ResponseEntity<ApiResponse> handleAccessDenied(RuntimeException e, HttpServletRequest req) {
    return ResponseEntity.status(HttpStatus.FORBIDDEN)
        .body(ApiErrorResponse.of(403, "FORBIDDEN", e.getMessage(), req.getRequestURI()));
}
```

B4 — user creat complet, un singur save, body întors

```
// Before — UserCommandServiceImpl.addUser
User user = User.builder()
    .firstName(...).lastName(...).email(...)
    .build(); // fără password/userType → 500 la insert
userRepository.save(user);
return UserMapper.toResponse(userRepository.save(user)); // save de 2 ori

// After
User user = User.builder()
    .firstName(...).lastName(...).email(...)
    .password(passwordEncoder.encode(userRequest.password()))
    .userType(userRequest.userType())
    .build();
user.setPermissions(permissionsForType(userRequest.userType()));
return UserMapper.toResponse(userRepository.save(user)); // un singur save
// iar în controller: return ResponseEntity.status(HttpStatus.CREATED).body(userResponse);
```

B5 — schemă gestionată, un singur nume de DB

```
# Before — application.yml
url: jdbc:mysql://127.0.0.1:3306/solarSyncBroker-api?...&createDatabaseIfNotExist=true
jpa.hibernate.ddl-auto: create
flyway.enabled: false

# After
url: jdbc:mysql://127.0.0.1:3306/solarsync?useSSL=false&serverTimezone=UTC
username: ${DB_USERNAME:solarsync}
jpa.hibernate.ddl-auto: validate
flyway.enabled: true # + V1__init.sql în db/migration
```

Puncte forte

- Ownership-ul pe `getHouseById` e acum legat corect de `Principal` — exact fix-ul cerut în pass-ul trecut.
- Vertical-slice curat, command/query separate, DTO-uri record, validare pe input.
- JWT stateless prin resource server + `@EnableMethodSecurity`; secret prin env var; `open-in-view: false`.
- 401/403 ies ca JSON consistent prin entry point + access denied handler — modelul de urmat și în `GlobalExceptionHandler`.
- Suita de teste crește constant (32 verzi) și IT-urile de securitate pe `house` (401/403) sunt exact genul de teste care lipsesc pe `battery`.

Ordine recomandată de reparare

1. **B1** — o linie de cod, cea mai mare gaură (oricine devine ADMIN).
2. **B2** — două string-uri; deblochează tot slice-ul `battery`. Aadaă imediat un IT `MockMvc` cu `authorities = {"battery:view"}` ca să rămână prins.
3. **B3** — erorile 404/409/403 devin JSON real; scenariul „user străin → 403” azi dă 500.
4. **B4** — scoate sau repară `/api/v1/user/add`.
5. **B5** — Flyway + `validate` înainte de orice date reale.
6. Apoi M1-M2 (ownership pe liste + battery), restul 🟡/🟢 ca pas de curățenie.

Q&A — verifică-ți înțelegerea

1. Token-ul JWT conține claim-ul `authorities: ["house:view", "battery:view"]`. De ce trece `hasAuthority('house:view')` dar pică `hasAuthority('BATTERY_VIEW')`, deși ambele "există" în `UserPermissions`? Ce anume compară Spring și cu ce?
2. De ce testul `BatteryQueryServiceTests.testGetBatteryByHouseId` e verde, deși endpoint-ul HTTP corespunzător întoarce 403 pentru absolut oricine? Ce strat sare testul și ce fel de test ar fi prins bug-ul?
3. La register, de ce nu e suficient să validezi `userType` cu `@NotNull` — ce diferență de principiu e între "câmpul e prezent și valid sintactic" și "clientul are voie să decidă această valoare"?

(răspunde-le în scris sau la următoarea sesiune, apoi spune „next” pentru pass-ul următor)